B.TECH 3RD-YEAR AI/ML PROJECT

AI-Based Indian Sign Language Alphabet Recognition and Voice Conversion System

TECHNICAL DESIGN REPORT (TDR)

System architecture, components, data flow, APIs, ML design and testing approach

Project scope: Recognition of ISL alphabet/fingerspelling classes A–Z from images or webcam frames, conversion of recognized letters into editable text, and English text-to-speech output.

Document status: Planning / proposed design. This document describes the intended system. It does not claim that implementation, model training, testing or deployment has already been completed.

Prepared for group project planning and development. Team members, institute details, guide name and submission date can be added by the group.

Document Control

Item	Details
Project title	AI-Based Indian Sign Language Alphabet Recognition and Voice Conversion System
Document	Technical Design Report (TDR)
Version	1.0 – Proposed technical design
Status	Planning draft; implementation details may change after verification
Target classes	ISL alphabet/fingerspelling A–Z

1. Technical Overview

The proposed application follows a frontend–backend–ML architecture. The React frontend manages user interaction, browser webcam access, image upload, text editing and speech synthesis. A FastAPI backend accepts images or frame data, validates inputs, applies the same preprocessing used during model training and calls a loaded TensorFlow/Keras CNN model. The backend returns the predicted class and confidence information as JSON.

The training pipeline is separate from live inference. It inspects and prepares the dataset, trains the CNN, evaluates it on held-out data and saves the model and evaluation artifacts. The inference backend loads the saved model rather than retraining it for every request.

2. High-Level Architecture

Browser / React UI → HTTP request → **FastAPI backend** → image validation and preprocessing → **TensorFlow/Keras CNN** → prediction JSON → **React UI** → text builder → browser English speech synthesis.

Component	Responsibility	Input / output
React frontend	Pages, camera permission, upload, display, text builder and controls.	User actions / API requests and responses.
Camera capture	Use browser getUserMedia and capture frames at a controlled rate.	Browser video frame / image blob.
FastAPI	Validate requests, route to preprocessing and model inference.	Image request / JSON prediction.
Preprocessing	Resize and format input exactly like training.	Image array / normalized tensor.
CNN model	Compute class scores for the 26 target labels.	Tensor / class scores.
SQLite	Optional record of prediction events and timestamps.	Structured records.
Speech synthesis	Speak user-approved text in English.	Text / browser audio output.

3. Proposed Repository Structure

```
Sign-Language-Alphabet-Recognition/
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■ ■■■ assets/
■ ■■■ package.json
■■■ backend/
■ ■■■ main.py
■ ■■■ routes/
■ ■■■ services/
■ ■■■ database/
■ ■■■ requirements.txt
■■■ training/
■ ■■■ inspect_dataset.py
■ ■■■ preprocess.py
■ ■■■ train_model.py
■ ■■■ evaluate_model.py
■■■ dataset/
■ ■■■ public/
```

```
■ ■■■ custom/
■ ■■■ processed/
■■■ saved_models/
■■■ results/
■■■ docs/
■■■ .gitignore
■■■ README.md
```

This is a proposed structure, not a claim that these files already exist. The team should create only the folders and files needed for the current phase. Avoid committing raw datasets, large generated artifacts, virtual environments or secrets unless there is a specific approved reason.

4. Component Design

4.1 Dataset inspection and preparation

- Read the extracted dataset and enumerate class folders.
- Check expected class names against the target A–Z mapping.
- Count images per class and identify unreadable files.
- Inspect dimensions, colour channels and representative samples.
- Record source, licence, download date, class mapping and split structure.
- Keep original data unchanged; write processed data to a separate location if materialization is needed.

4.2 Preprocessing module

The training and inference code must share the same preprocessing rules. The initial proposal is to resize images to 128 × 128, maintain a consistent RGB channel order and normalize pixel values to the 0–1 range. Any change to image size or colour handling must be applied consistently during training and inference.

- Training-only augmentation may include small shifts, rotations or zooms if they preserve the meaning of the sign.
- Do not apply augmentation to validation or test data.
- Do not use horizontal flipping automatically; it may change or invalidate a sign depending on the label convention.
- Validate tensor shape and data type before model inference.

4.3 CNN model

The baseline model will use TensorFlow/Keras. A simple sequential CNN is sufficient for the first experiment: convolution and activation blocks, max-pooling, flattening, one or more dense layers, and a 26-unit softmax output layer. The model will be trained using a suitable optimizer such as Adam and a multi-class loss such as categorical or sparse categorical cross-entropy, depending on label encoding.

The architecture, number of filters, epochs, batch size and learning rate are experimental settings. They should be recorded after the first baseline run rather than presented as already optimized.

4.4 Model output and confidence

The model returns scores for the configured class order. The backend selects the highest-scoring class and may return its score as a confidence-like value. Softmax scores are not guaranteed to be calibrated probabilities. The UI should avoid implying certainty and should allow low-confidence predictions to be ignored or reviewed.

5. Dataset and Label Mapping

The target label list is A through Z. A single source of truth for class order must be maintained and shared by training and inference. Do not rely on an accidental filesystem order unless it is explicitly sorted and saved. Store the label mapping with the model or in a versioned configuration file.

Data split	Use	Rule
Training	Fit model weights.	Only training data is augmented.
Validation	Monitor training and compare settings.	Not used as final performance claim.
Testing	Final evaluation.	Keep separate until final evaluation.

Data split	Use	Rule
Custom holdout	Check generalization to newly collected signers.	Do not use for tuning if claiming an independent test.

The public dataset documentation reports multiple signers, but the actual split must be inspected before claiming signer-independent evaluation. If source data does not support that split, report the limitation clearly.

6. Training Pipeline Design

- Load dataset paths and labels from the verified folder structure.
- Validate class mapping and count images by split.
- Decode images and apply shared preprocessing.
- Create training batches and validation batches.
- Build and compile the CNN.
- Train while recording training and validation loss/accuracy.
- Save the trained model and label mapping.
- Save training history and configuration for reproducibility.

7. Evaluation Design

The evaluation module will run the saved model against the held-out test dataset. It will generate overall accuracy, per-class precision/recall/F1, a confusion matrix and training history plots. The report should include test-set size, class distribution, split method and known limitations. Results must be generated from actual runs.

Metric / artifact	What it tells us
Accuracy	Fraction of test examples classified correctly.
Precision	Among predictions for a class, how many were correct.
Recall	Among actual examples of a class, how many were found.
F1-score	Harmonic balance of precision and recall.
Confusion matrix	Which classes are being confused with other classes.
Learning curves	Training and validation behaviour; can reveal overfitting or underfitting.

8. Backend API Design

The following endpoint names are proposed for planning. Exact request fields and response schemas should be finalized during backend implementation and documented in the API. All prediction routes should return clear validation errors rather than crashing on unsupported input.

Method	Endpoint	Purpose
GET	/health	Report whether the API and model service are available.
GET	/classes	Return the configured alphabet labels.
POST	/predict	Accept an uploaded image and return the predicted class and score.
POST	/predict-frame	Accept a webcam frame and return the predicted class and score.
GET	/metrics	Return saved evaluation metrics when they exist.
GET	/history	Return stored prediction history if SQLite history is implemented.

8.1 Example prediction response (illustrative schema)

```
{
  "predicted_class": "H",
  "confidence": 0.91,
  "model_available": true
}
```

The values above are an example of response structure only. They are not a real model prediction or measured confidence result.

9. Frontend Design

Page / module	Main responsibilities
Home	Explain the project, scope and navigation.
AI Recognition	Image upload, webcam preview, prediction result, confidence and text builder.
Dashboard	Show actual metrics and optional prediction history.
Alphabet Learning	Display verified A–Z reference material.
About	Explain objective, stack, limitations and project details.
Model Performance	Show real accuracy/loss plots, confusion matrix and classification report.

10. Webcam and Gesture-to-Text Flow

- Request camera access from the browser using getUserMedia after user action.
- Show the camera preview and capture frames at a controlled interval.
- Send frames to the backend as image data using the selected API contract.
- Backend validates, preprocesses and runs inference.
- Frontend displays the returned label and score.
- Use stable-prediction logic and a cooldown or explicit Add Letter action to prevent repeated letters from consecutive frames.
- Allow the user to review and edit the text before speaking.

The browser owns camera permission and capture. The backend receives frame data; it does not automatically gain direct access to the user's webcam.

11. Text-to-Speech Design

Use the browser SpeechSynthesis API to speak the text builder's current text in English. Speech should start only after the user presses Speak. Provide a Stop control and handle browsers where speech synthesis or an English voice is unavailable. This is text-to-speech, not speech recognition.

12. Optional SQLite Data Design

SQLite is intended only for lightweight application records such as prediction history. It is not the training dataset and does not replace the image folders or saved model artifacts. If implemented, a simple prediction_history table may contain an id, predicted_label, confidence_score, created_at and input_source. Avoid storing raw user images unless there is a clear need and user consent.

13. Error Handling and Security

- Reject unsupported file formats and empty uploads with a readable error response.
- Set reasonable image size limits and validate decoded image dimensions.
- Handle missing model files with an explicit model-unavailable status.
- Do not return stack traces or local filesystem paths to the browser.
- Keep secrets in environment variables and out of Git.
- Restrict CORS to the development frontend origin where practical.
- Avoid saving personal images by default; document any storage behaviour.
- Do not use mock predictions as if they came from the trained model.

14. Git and Version-Control Design

Use small commits after a logical task is implemented and tested. Keep the main branch in a runnable state where practical. Use descriptive commit messages and review the diff before committing. Do not push to a remote repository without team approval.

- chore: initialize project structure
- chore: configure python environment
- data: add dataset inspection script
- feat: implement image preprocessing
- feat: build baseline cnn model
- feat: add model evaluation
- feat: create prediction api
- feat: build recognition interface
- feat: integrate webcam prediction
- feat: add english text to speech
- docs: add setup and project documentation

The .gitignore should exclude virtual environments, local .env files, caches, temporary uploads, large raw datasets and generated files unless the team explicitly decides to version a small artifact. Track dataset instructions and metadata instead of committing large datasets by default.

15. Testing Strategy

Test level	What to verify
Dataset checks	Class folders, labels, counts, unreadable images, dimensions and split separation.
Preprocessing checks	Expected tensor shape, colour order, normalization range and consistent inference preprocessing.
Model checks	Model loads, output dimension is 26, predictions map to the correct label order.
API checks	Valid image, invalid file, empty upload, missing model and unexpected input.
Frontend checks	Upload preview, loading/error states, prediction display and text controls.
Webcam checks	Permission denial, frame capture, API response and repeated-prediction handling.
End-to-end checks	Gesture/image → predicted letter → editable text → English speech.
Generalization checks	Different people, backgrounds, lighting, distances and camera conditions.

16. Deployment and Run Configuration

During development, the frontend and backend may run as separate local services. The exact ports and commands will be recorded in README after the environment is selected. The frontend will call the backend using a configurable API base URL. Production deployment is optional and should be attempted only after local integration works.

- Use a Python virtual environment for backend and training dependencies.
- Keep frontend dependencies managed through the package manager and package.json.
- Record tested Python and package versions.
- Keep model path and API URL configurable rather than hard-coded in many files.
- Document camera permission requirements; browser camera access generally requires a secure context such as localhost or HTTPS.

17. Technical Dependencies and Risks

- TensorFlow compatibility depends on Python version and operating system; verify official installation support before choosing versions.
- The model's performance depends on dataset quality, label consistency and signer variation.
- Static-image CNNs cannot directly model temporal movement; motion-dependent signs may need a separate sequence model.
- Browser webcam frame rate and image size affect network load and perceived latency.
- Training hardware may limit batch size and model complexity; start with a small baseline.
- Frontend and backend preprocessing must remain identical to training preprocessing.

18. Implementation Sequence

Sequence	Technical output / gate
1. Setup	Python/VS Code environment, repository, .gitignore and folder skeleton verified.
2. Dataset audit	Actual archive structure, labels, counts and licence recorded.
3. Preprocessing	Preprocessing script produces valid model-ready batches.
4. CNN baseline	Model trains, saves and reloads successfully.
5. Evaluation	Test metrics and plots generated from held-out data.
6. API	Prediction API loads saved model and passes API tests.
7. UI	React pages and image upload work.
8. Integration	Webcam, text builder and TTS work end-to-end.
9. Finalization	Regression testing, README, report, PPT and viva material completed.

19. Design Decisions to Confirm During Implementation

- Exact image dimensions and preprocessing configuration after inspecting dataset samples.
- Whether all A–Z classes in the chosen source are valid still-image classes.
- Final CNN layer sizes and training hyperparameters after baseline experiments.
- Whether webcam recognition adds letters automatically or requires an explicit Add Letter action.
- Final API request format for webcam frames.
- Whether SQLite history is needed within the available project schedule.
- Whether the team will deploy online or demonstrate locally.

20. References

- TensorFlow documentation: https://www.tensorflow.org/
- Keras documentation: https://keras.io/
- OpenCV documentation: https://docs.opencv.org/
- FastAPI documentation: https://fastapi.tiangolo.com/
- React documentation: https://react.dev/
- MDN – MediaDevices getUserMedia: https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia
- MDN – Web Speech API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
- RealSign ISL dataset repository: https://github.com/RealSign62/RealSign-Indian-Sign-Language-Dataset

This TDR is a proposed technical design. Update it to match the implementation actually built, tested and approved by the project team. Do not leave proposed components described as completed features in the final submission.